

Progetto 1: Analisi di dati sui voli con Apache Spark

Sistemi e Architetture per Big Data - A.A. 2025/26

Luca Carloni
0366574

Alessandro Vassallo
0366572

Andrea Galluzzi
0365026

Sommario—Il presente lavoro propone un’architettura di batch processing, incentrata sul framework Apache Spark, per l’analisi di un dataset di circa 2,2 milioni di eventi relativi ai voli aerei statunitensi (gennaio-aprile 2025). La fase di data ingestion, gestita tramite Apache NiFi, converte le sorgenti CSV nel formato colonnare Parquet e ne attua il partizionamento per compagnia aerea su HDFS, abilitando meccanismi di partition pruning e column pruning. Su tale base, tre query analitiche vengono implementate mediante le API DataFrame, SQL e RDD, di cui si confrontano le prestazioni; per la stima dei percentili si introducono inoltre varianti approssimate fondate su strutture sketch (t-digest e KLL). L’analisi è completata da un clustering K-Means delle compagnie aeree. La valutazione sperimentale, basata su esecuzioni ripetute, quantifica i compromessi tra le diverse strategie di elaborazione.

Index Terms—Apache Spark, Apache NiFi, Apache Parquet, Apache HDFS, Redis, Grafana

I. INTRODUZIONE

L’obiettivo del progetto è di sviluppare un’architettura di batch processing, con particolare attenzione al framework di data processing **Apache Spark**, per rispondere ad alcune query su dati storici relativi a voli aerei statunitensi, forniti dal Dipartimento dei trasporti degli Stati Uniti d’America. L’architettura permette di trasformare i dati grezzi in risultati interpretabili, utili per analizzare il comportamento delle compagnie aeree. Il dataset contiene circa 2.200.000 eventi, che descrivono i voli effettuati nel periodo dal 1 gennaio 2025 al 30 aprile 2025. La quantità di dati forniti non giustificano appieno l’utilizzo di framework Big Data, ma le scelte progettuali e di sviluppo sono state guidate dalla volontà di simulare un contesto dove è necessario avere questo tipo di approccio.

II. ARCHITETTURA DEL SISTEMA

L’architettura utilizzata è composta da framework orientati ai Big Data e organizzata secondo una pipeline di lavoro ben specifica. È possibile osservare la topologia dell’architettura in Figura.

III. DATA INGESTION

Per gestire l’ingestione dei dati relativi ai voli da gennaio ad aprile delle compagnie aeree statunitensi all’interno del nostro sistema, rendendoli contestualmente processabili da Apache Spark, è stato necessario fare delle attente valutazioni sulla maniera più opportuna per memorizzare i dati stessi all’interno

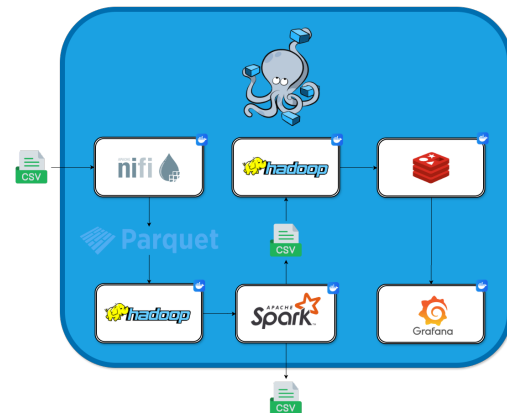


Figura 1. Architettura del sistema

dell’architettura e utilizzare il framework Apache NiFi per effettuare il preprocessing opportuno ai nostri intenti.

A. Apache Parquet

Il dataset è stato fornito in diversi file in formato CSV, un file per ciascun mese di osservazione. CSV è un formato scomodo per l’elaborazione analitica su grandi volumi di dati, non essendo tipizzato e non offrendo una compressione dei dati efficiente. Dopo attente valutazioni la scelta è ricaduta sul formato **Apache Parquet**, un formato colonnare progettato per storage e recupero efficiente, diventato di fatto uno standard all’interno degli ecosistemi Big Data. Parquet infatti è più veloce e occupa meno spazio di altri formati, attuando una filosofia di column pruning, per cui permette di leggere solo le colonne richieste. La scelta è ricaduta su Parquet anche per la natura stessa del progetto, trattandosi di un formato molto più efficiente in lettura che in scrittura, dove magari si sarebbe comportato meglio un formato come Apache Avro. È importante sottolineare il criterio con cui i file del dataset originale, forniti come CSV separati per mese, sono stati convertiti e organizzati su HDFS: la conversione trasforma i file in formato Parquet e li **partiziona** fisicamente per compagnia aerea, adottando una struttura a directory in *stile Hive* del tipo `/sabd/processed/OP_UNIQUE_CARRIER=<carrier>`. La scelta della chiave di partizionamento non è casuale, ma

ha lo scopo principale di sfruttare il **partition pruning**: la capacità di Spark di leggere unicamente le directory pertinenti a una certa query, ignorando le restanti. Poiché tutte le interrogazioni del progetto selezionano sottoinsiemi di compagnie (infatti la Q1 filtra i vettori AA e DL, la Q3 i vettori AA, DL, UA e WN) il partizionamento per carrier consente a Spark di accedere alle sole cartelle d'interesse, riducendo concretamente il volume di dati letti da HDFS. Il partizionamento per mese, che avrebbe mantenuto la suddivisione temporale originaria, non trova riscontro con la natura stessa delle query, poiché nessuna delle tre query filtra per uno specifico mese o intervallo temporale, e il beneficio del pruning non si sarebbe mai concretizzato. I singoli file Parquet risultano inferiori alla dimensione di default dei blocchi HDFS 3.x (128 MB) e ciò andrebbe a introdurre un certo grado di frammentazione, a causa dell'inutilizzo di ciascun blocco dove viene salvato ciascun file. L'unico costo introdotto è l'overhead a carico del Namenod dovuto ai metadati, ma data la dimensione contenuta del dataset, tale overhead è del tutto trascurabile rispetto al guadagno di I/O garantito dal partition pruning.

B. Apache NiFi

NiFi è un sistema per l'automatizzazione del flusso di dati tra sistemi, basato sul concetto di programmazione flow-based e usato principalmente per il routing e la trasformazione dei dati. La scelta di NiFi è dettata dalla versatilità, essendo compatibile con molti formati di dati (CSV, Apache Parquet, Apache Avro, ...) e dalla sua piena compatibilità con sistema di data storage, quali HDFS. In particolare NiFi è stato utilizzato per recuperare le sorgenti di dati in formato CSV e convertirle in formato Apache Parquet e andando contestualmente a rimuovere le colonne superflue. Una volta processati, i dati vengono salvati in maniera persistente in HDFS, in modo tale da renderli disponibili al data processing tramite Apache Spark.

1) *NiFi Flow*: Il flusso gestisce l'ingestione dei dati dai file sorgente fino a HDFS, con una fase di preprocessing. La pipeline è composta da cinque processori in sequenza:

- 1) **ListFile** monitora la directory locale in modo ricorsivo e genera un FlowFile per ogni file il cui nome soddisfa una certa espressione regolare, selezionando così i quattro file mensili di reporting (*gennaio-aprile 2025*). Il listing permette un'ingestione singola, per cui i file già elencati non vengono rielaborati. Il FlowFile prodotto contiene solo i metadati del file, non ancora il contenuto.
- 2) **FetchFile** legge il contenuto effettivo di ciascun file sfruttando i metadati recuperati dal Processor precedente e lo carica nel FlowFile.
- 3) **PartitionRecord** è il cuore della trasformazione e assolve due compiti contemporaneamente. PartitionRecord lavora sempre con due controller service: un Record Reader (lettura) e un Record Writer (scrittura). In *lettura* adotta un **CSVReader** e in *scrittura* un **ParquetRecordSetWriter**, realizzando la **conver-**

sione da CSV a Parquet. Contestualmente **ripartiziona i record** in base al valore della compagnia aerea (OP_UNIQUE_CARRIER), producendo un FlowFile Parquet distinto per ciascun vettore presente nel file mensile. La **selezione delle colonne** avviene proprio in questa fase: il CSVReader impiega uno *schema Avro* che mantiene le sole 14 colonne utili al progetto, scartando tutte le restanti colonne della sorgente. Lo schema di scrittura ne conserva 13, omettendo OP_UNIQUE_CARRIER, poiché tale informazione viene promossa a chiave di partizionamento e codificata nel percorso delle directory, evitando così la ridondanza del dato. Il tipo OP_UNIQUE_CARRIER, che è una stringa, verrà inferita automaticamente da Spark quando andrà a fare partition discovery, seguendo lo standard Hive.

4) **UpdateAttribute** imposta l'attributo filename del FlowFile risultante assegnando un nome significativo a ciascun file Parquet.

5) **PutHDFS** salva infine i file Parquet su HDFS. Al termine, i dati sono disponibili in modo persistente su HDFS per il processing tramite Apache Spark.

2) *Schema Avro*: Come già introdotto, PartitionRecord utilizza un CSVReader e un ParquetRecordSetWriter, entrambi configurati con uno schema Avro che ne definisce la struttura e la tipizzazione dei campi, poi convertiti in formato Parquet. Lo schema Avro è di fatto un documento JSON usato in NiFi per descrivere i dati. In particolare:

Tabella I
SCHEMA AVRO DEI CAMPI DEL DATASET.

Campo	Tipo Avro
YEAR, MONTH, DAY_OF_MONTH, CRS_DEP_TIME	["null", "int"]
OP_UNIQUE_CARRIER	["null", "string"]
DEP_DELAY, ARR_DELAY, CANCELLED, DIVERTED, CARRIER_DELAY, WEATHER_DELAY, NAS_DELAY, SECURITY_DELAY, LATE_AIRCRAFT_DELAY	["null", "double"]

3) *Automatizzazione del flow attraverso REST API*: L'intero ciclo di vita del flusso è automatizzato tramite uno script Python (*start_nifi_flow.py*) che dialoga con le **REST API** esposte da NiFi, eliminando ogni intervento manuale sulla UI e rendendo l'ingestione interamente riproducibile all'avvio dell'ambiente.

IV. STORAGE

A. Apache Hadoop Distributed File System (HDFS)

Apache Hadoop Distributed File System (HDFS) è un file system distribuito open source, un'ottima soluzione per applicazioni batch con file di grandi dimensioni. Si tratta di un sistema di storage persistente basato su un'architettura Master/Worker, dove si ha un Namenode e molteplici Datanode, nella nostra architettura si hanno 2 Datanode. I file memorizzati in HDFS vengono suddivisi in blocchi, con dimensione di

default pari a 128 MB. HDFS supporta *Partition tolerance* per cui è possibile impostare un fattore di replicazione, nel nostro caso pari a 1. Oltre a inserire il dataset preprocessato, HDFS viene anche sfruttato per mantenere i risultati delle query in CSV una volta eseguite dai job di Spark.

B. Remote Directory Server (Redis)

Come richiesto dalla specifica del progetto, una volta caricati i risultati delle query in HDFS occorre anche esportarli a partire da quest'ultimo verso un sistema di storage. Non tutti i risultati delle query sono dati a serie temporali (o time series), per cui la scelta non è ricaduta in maniera ovvia su database a serie temporali come **InfluxDB**. Un aspetto che si è deciso di tenere in considerazione durante lo sviluppo dell'architettura è che questo sistema di storage avrebbe assunto anche il ruolo di **sorgente per la generazione dei grafici** per la visualizzazione dei risultati delle query, sfruttando il framework **Grafana**. Mossi da quest'ultimo aspetto, la scelta del sistema di storage è ricaduta su **Remote Directory Server (Redis)**, uno tra i più popolari data store key-value, con la peculiarità di essere un data store in-memory, per cui molto veloce. La rapidità e la sua estrema compatibilità con Grafana ci hanno guidato nella sua scelta, motivata inoltre dalla natura dei dati esportati: non si tratta del dataset grezzo dei voli, ma di risultati già aggregati e di dimensioni ridotte, prodotti al termine delle elaborazioni di Spark. Un'alternativa che abbiamo preso in considerazione e analizzato è **HBase**, un'implementazione open-source di Bigtable sopra HDFS, che abbiamo ritenuto essere una soluzione overkill per la semplice memorizzazione dei risultati delle query.

V. DATA PROCESSING

Terminata la fase di Data Ingestion, il passo successivo nella pipeline è quello di rispondere ai quesiti posti dalla specifica con il processamento dei dati tramite query.

A. Apache Spark

Apache Spark è un engine di analisi per il processamento di dati su larga scala, che permette l'esecuzione di job in maniera completamente distribuita e parallelizzata. Spark espone API di diverso tipo costruite su Spark Core, come Spark SQL per query in formato SQL-like e Spark MLlib che implementa i principali algoritmi di Machine Learning. Spark Core fornisce anche l'astrazione dati principale: **Resilient Distributed Dataset (RDD)**, collezione di elementi distribuiti sui nodi del cluster ed elaborati in parallelo. Nella nostra architettura il Cluster Manager utilizzato è il manager built-in di Spark, **Standalone**.

B. Query

Di ciascuna delle tre query sono state create tre diverse implementazioni che sfruttano API differenti (**DataFrame**, **SQL** e **RDD**), in modo da analizzare le performance di ciascun approccio. Alla fine di ogni query, come già spiegato nella sezione precedente, il risultato viene salvato in formato CSV

all'interno di HDFS. Questa sezione sarà limitata semplicemente alla versione RDD delle query, ritenendola la più interessante da analizzare.

C. Struttura comune delle query RDD

A differenza delle versioni DataFrame e Spark SQL, nelle implementazioni RDD il calcolo è espresso direttamente come una pipeline di trasformazioni e azioni, dove le trasformazioni sono **lazy**, per cui Spark costruisce unicamente il piano di esecuzione (lineage) e nulla viene calcolato finché non scatta un'azione. Ogni query parte quindi da una lettura colonnare del dataset in Parquet sulla quale si applicano **column pruning** (selezione delle sole colonne utili) e **predicate pushdown** (filtri più vicini all'operazione di lettura dalla sorgente dati). Solo a valle di queste operazioni si passa all'API RDD (con .rdd) andando a sfruttare l'efficienza del formato Parquet, pur restando nel paradigma RDD richiesto. Quando un medesimo RDD va ad alimentare più azioni, viene cachato (con cache) per evitarne il ricalcolo contestualmente a ogni azione. I risultati vengono infine salvati sul cluster HDFS e successivamente esportati su Redis per la visualizzazione in Grafana.

D. Studio dell'esecuzione delle query

Il job principale per lo studio dell'esecuzione delle query è uno strumento per misurare i tempi di esecuzione delle query Spark. Il modulo esegue una stessa query per diverse run (di default 100), cronometrando ogni esecuzione, per poi calcolare le statistiche relative a uno studio di una certa query e salvare i tempi all'interno di un file CSV. Ciò permette anche una strutturazione del codice di ciascuna query, a prescindere dall'API utilizzata, in maniera pulita e disaccoppiata dallo studio dei tempi di esecuzione.

1) *Query 1*: La prima query calcola per le sole compagnie AA e DL delle statistiche aggregate su base mensile, quindi per ogni coppia (*anno, mese*): il numero totale di voli, il numero di voli cancellati, il cancellation rate, media, minimo e massimo di DEP_DELAY. Il tasso di cancellazione è calcolato su **tutti** i voli considerando anche i voli cancellati o deviati, mentre le statistiche sul ritardo considerano solo i **voli non cancellati**.

- 1) Legge il dataset dal cluster HDFS con spark.read.parquet, selezionando solo le 5 colonne utili e passa quindi all'RDD con .rdd
- 2) Mantiene i soli voli delle compagnie AA e DL con filter.
- 3) Con map trasforma ogni record nella struttura con chiave (carrier, year, month) e valore (1, cancelled, delay_sum, delay_n, delay_min, delay_max). In questa un volo non cancellato e con ritardo valido contribuisce con il proprio DEP_DELAY, mentre i voli cancellati o privi di ritardo non contribuiscono
- 4) Con reduceByKey aggrega i valori parziali per chiave, sommando i voli totali, i voli cancellati, i contributi dei ritardi e contando i voli in ritardo e valutando il ritardo minimo e massimo

- 5) Con mapValues applica la funzione di finalizzazione che calcola le metriche definitive (cancellation rate, media, min, max) senza alterare le chiavi
 - 6) Il risultato viene cachata, dato che verrà riusato da due azioni consecutive
 - 7) Con collect materializza il risultato sul driver
 - 8) Infine salva i risultati sia su CSV locale sia sul cluster HDFS. I dati vengono poi caricati su Redis per i grafici Grafana.
- 2) *Query 2*: La seconda query calcola per ogni compagnia con almeno 500 voli non cancellati e non deviati: il numero di voli, la media di ARR_DELAY e la media delle cinque componenti o causa di ritardo, restituendo la *top-10* delle compagnie ordinate per ritardo medio in arrivo decrescente. I valori NULL nelle componenti di ritardo sono trattati come 0, perché nel dataset originale le componenti di ritardo sono state riportate NULL quando il ritardo totale è inferiore a 15 minuti; ciò è equivalente a dire che quelle componenti hanno un contributo pari a 0 nel calcolo del loro stesso valore medio, ma i relativi voli vanno comunque considerati nel conteggio totale per il calcolo di quest'ultimo.
- 1) Legge il dataset Parquet da HDFS applicando già in lettura il filtro CANCELLED = 0 e DIVERTED = 0, seleziona le 7 colonne utili e passa all'RDD con .rdd
 - 2) Con map trasforma ogni record nel formato (*compagnia, (il conteggio dei voli, la somma e il numero di ARR_DELAY validi e la somma di ciascuna delle cause di ritardo)*), dove le cause a NULL vengono convertite a 0
 - 3) Con reduceByKey aggrega i valori parziali per compagnia, sommando tutte le componenti
 - 4) Con mapValues calcola le medie per compagnia (media ARR_DELAY e media delle cinque cause), dividendo per il numero di voli validi
 - 5) Con filter mantiene le sole compagnie con almeno 500 voli;
 - 6) Con l'azione takeOrdered estrae direttamente la top-10 per ARR_DELAY medio decrescente. Ogni partizione conserva solo i propri primi 10 elementi, che vengono poi stabiliti in ultima istanza sul driver
 - 7) Poiché takeOrdered è un'azione che restituisce una lista, il risultato viene riconvertito in RDD con parallelize per il salvataggio in stile RDD
 - 8) Infine salva i risultati sia su CSV locale sia sul cluster HDFS. I dati vengono poi caricati su Redis per i grafici Grafana.
- 3) *Query 3*: La terza query analizza le compagnie AA, DL, UA e WN suddividendo i voli in 24 fasce orarie (l'ora è derivata da CRS_DEP_TIME). Per ogni coppia (*compagnia, fascia oraria*) calcola i percentili 25/50/75/90 di DEP_DELAY (sui voli non cancellati) e per ogni compagnia calcola il minimo e il massimo di DEP_DELAY sull'intero dataset. Poiché il calcolo dei percentili è oneroso, andiamo a proporre una versione esatta, ma anche due versioni approssimate basate su strutture sketch: **KLL** e **t-digest**. Le strutture sketch vengono costruite

su partizioni diverse di cui è possibile fare il *merge*, cioè combinandole in ultima istanza in unico sketch sul driver, ottenendo un risultato approssimativo, ma quasi identico a quello che si otterrebbe costruendolo su tutti i dati.

- 1) Legge il dataset Parquet da HDFS filtrando le compagnie AA, DL, UA, WN e i voli non cancellati, deriva la colonna delle ore prendendo la parte intera inferiore di CRS_DEP_TIME/100 e seleziona le 3 colonne utili, elimina i DEP_DELAY nulli e passa all'RDD
 - 2) Con map costruisce il record nel formato (*compagnia, ora, ritardo*), che viene cachato perché riusato dai due rami successivi
 - 3) Ramo percentili:
 - a) Con map trasforma il record in (*compagnia, ora, delay*), definendo la chiave
 - b) Con groupByKey raggruppa tutti i ritardi per (*compagnia, ora*)
 - c) Con mapValues ordina i valori del gruppo e calcola i percentili 25/50/75/90 in modo esatto con interpolazione lineare
 - d) Con map trasforma il record in (*compagnia, ora, p25, p50, p75, p90*)
 - e) Con sortBy ordina per (*compagnia, ora*)
 - 4) Ramo min/max:
 - a) Con map trasforma il record in (*compagnia, (delay, delay)*)
 - b) Con reduceByKey calcola minimo e massimo per compagnia
 - c) Con sortByKey ordina per compagnia
 - 5) Cache di entrambi gli RDD risultanti e materializzazione con collect
 - 6) Infine salva entrambi i risultati sia su CSV locale sia sul cluster HDFS. I dati vengono poi caricati su Redis per i grafici Grafana.
- a) *Versioni efficienti dei percentili (t-digest e KLL)*: La versione esatta appena presentata usa groupByKey, che porta in shuffle tutti i valori di ciascun gruppo e li materializza su un singolo nodo: è costoso in memoria. Per soddisfare il requisito di tecniche efficienti per il calcolo dei percentili, sono state realizzate due varianti RDD che sostituiscono groupByKey con combineByKey e aggregano i dati in strutture sketch compatte e di cui è possibile fare il *merge*, senza mai collezionare tutti i valori grezzi per gruppo:
- **t-digest**: combineByKey costruisce un TDigest per chiave (*create_combiner*), lo aggiorna con i valori della stessa partizione (*merge_value*) e fonde i digest tra partizioni trasferendo i soli centroidi (*merge_combiners*)
 - **KLL**: stesso schema combineByKey del caso precedente, ma con sketch KLL della libreria Apache DataSketches, dove però lo sketch viene serializzato in bytes per compatibilità con la serializzazione pickle di Spark.
- In entrambe le varianti il calcolo dei percentili è incrementale e distribuito, riducendo drasticamente shuffle e occupazione di

memoria rispetto alla versione esatta. Il confronto dei tempi tra versione esatta, t-digest e KLL è riportato nella Sezione *Valutazione dei Risultati*.

E. Clustering K-Means

Il clustering ha lo scopo di fornire una visione sintetica e comparativa delle compagnie aeree presenti nel dataset. Si tratta di un'analisi che mira a individuare gruppi di compagnie con comportamenti operativi simili. Ogni compagnia viene rappresentata tramite un vettore di feature numeriche e tali vettori vengono successivamente confrontati mediante l'algoritmo K-Means, che assegna le compagnie caratterizzate da profili simili in termini di ritardi, cancellazioni e cause di ritardo a uno stesso cluster. In questo caso l'aggregazione rappresenta un passaggio assolutamente necessario per costruire l'unità di analisi del clustering, dato che l'algoritmo non viene applicato ai singoli voli, ma ai profili aggregati delle compagnie aeree. Ogni carrier viene quindi descritto da un'unica osservazione, ottenuta calcolando sul periodo analizzato le feature operative selezionate, come ritardi medi, tasso di cancellazione e contributi medi delle diverse cause di ritardo. La specifica suggerisce di considerare le compagnie aeree con il maggior numero di voli, ma nel nostro caso l'analisi di clustering è stata eseguita su tutte le compagnie disponibili.

1) *Organizzazione dell'analisi tramite studi*: Abbiamo deciso di stata organizzare l'analisi secondo due *studi distinti* e confrontabili: uno studio **base**, che utilizza esclusivamente le feature richieste dalla specifica progettuale, e uno studio **esteso**, che invece aggiunge ulteriori metriche operative tramite *feature engineering*, con l'obiettivo di valutare se informazioni aggiuntive possano migliorare l'individuazione di compagnie operativamente simili tra di loro.

Queste feature descrivono il comportamento operativo delle compagnie aeree rispetto ai principali aspetti richiesti dalla traccia: puntualità, cancellazioni e composizione delle cause di ritardo. Nel calcolo dei ritardi medi e delle cause di ritardo sono stati considerati solo i voli non cancellati; per il ritardo medio in arrivo sono stati esclusi anche i voli dirottati, poiché non rappresentano un completamento ordinario della tratta. I valori nulli relativi alle cause di ritardo sono stati sostituiti con zero, interpretando l'assenza di una specifica causa come contributo nullo al ritardo.

Lo studio esteso aggiunge tre ulteriori feature:

- La **deviazione standard del ritardo in partenza** (`std_dep_delay`) che consente di misurare la variabilità del comportamento della compagnia.
- Il **tasso dei voli arrivati in tempo** (`on_time_rate`) che fornisce una misura diretta di quanto una compagnia sia operativamente regolare.
- Il **tasso dei voli deviati** (`diverted_rate`) che permette di includere nel profilo della compagnia anche la quota

di voli deviati, comportamento anomalo diverso dalla cancellazione, già considerata.

2) *Selezione delle feature aggiuntive tramite matrice di correlazione (studio esteso)*: Prima di eseguire il clustering nella configurazione estesa, abbiamo calcolato la matrice di correlazione di Pearson tra tutte le feature candidate, quindi le feature di base insieme alle tre feature aggiuntive. La matrice di correlazione è stata valutata sui valori aggregati per compagnia area. L'obiettivo di quest'analisi è di evitare di introdurre variabili ridondanti: due feature fortemente correlate descrivono la stessa informazione e, nello spazio standardizzato del K-Means, ne vanno a distorcere le distanze euclidee e quindi la formazione dei cluster. Per ciascuna delle tre feature aggiuntive è stata considerata la correlazione massima in valore assoluto con una qualsiasi delle altre feature e se superata la soglia di ridondanza $|r| \geq 0.80$, la feature viene considerata ridondante scartata. L'analisi ha prodotto i seguenti esiti:

- `std_dep_delay` risulta **ridondante**: presenta correlazione 0.92 con `avg_dep_delay`, 0.89 con `avg_arr_delay` e 0.82 con `avg_carrier_delay`. Ciò significa che la variabilità del ritardo in partenza è già ben spiegata dal ritardo medio. Viene quindi rimossa.
- `on_time_rate` viene **mantenuta**: la sua correlazione massima è 0.41 con `avg_late_aircraft` e mostra correlazioni negative ma moderate con i ritardi (-0.39 con in partenza, -0.35 in arrivo), coerenti dal punto di vista operativo, ma sotto soglia. Quindi informazione non ridondante.
- `diverted_rate` viene **mantenuta**: correlazione massima 0.52 (con `avg_weather_delay`) e 0.43 con `avg_carrier_delay`. Valori moderati che indicano la presenza di informazione in parte nuova, legata alle interruzioni operative.

A valle di quest'analisi lo studio esteso utilizza 10 feature. Questo passaggio inoltre garantisce che le componenti aggiuntive inserite nel modello aumentino effettivamente il contenuto informativo del clustering rendendo effettivamente più robusta e interpretabile l'analisi dei cluster.

3) *Preprocessamento*: Le feature selezionate per ciascuno studio vengono combinate in un unico vettore numerico di feature tramite **VectorAssembler**, a partire dai profili aggregati delle compagnie. Ciò è necessario perché gli algoritmi della libreria **MLlib** di Spark utilizzano come input una colonna vettoriale contenente le variabili del modello. Il vettore viene successivamente *standardizzato* tramite **StandardScaler**, che va a impostare media pari a 0 e deviazione standard pari a 1, dato che l'algoritmo K-Means si basa sul calcolo di distanze euclidee, perciò le feature devono essere espresse sulla stessa scala numerica, perché altrimenti scale numeriche diverse andrebbero a influenzare diversamente l'assegnazione ai cluster, prescindendo dalla loro effettiva importanza.

4) *Scelta dell'iperparametro k* : La scelta di k è stata supportata da due metriche:

- Il **Metodo Elbow** basato sull'andamento del **WSSSE (Within Set Sum of Squared Error)**. Il WSSSE misura quanto le osservazioni assegnate a uno stesso cluster risultino vicine al rispettivo centroide. Valori più bassi indicano cluster più compatti. Tuttavia occorre considerare che il WSSSE tende a diminuire all'aumentare di k . Questo metodo viene utilizzato per scegliere k osservando l'eventuale punto di gomito nella curva, cioè si sceglie il k in cui l'errore diminuisce di più prima di rallentare l'entità della sua discesa.
- Il **Silhouette Score** valuta la **coesione interna dei cluster** e la **separazione rispetto agli altri cluster**: valori più elevati indicano una migliore qualità del clustering.

Abbiamo limitato $k \in [2, 6]$, così da confrontare diverse possibili suddivisioni delle compagnie aeree: valori superiori di k non sono stati considerati perché avrebbero potuto produrre cluster eccessivamente piccoli, riducendo l'utilità del clustering. Il valore finale di k viene scelto selezionando quello che **massimizza il Silhouette Score** nell'intervallo analizzato, utilizzando il Metodo Elbow più come un ulteriore supporto interpretativo. La Silhouette viene massimizzata in entrambi gli studi a $k = 2$ (0.584 nello studio base, 0.384 nello studio esteso con feature aggiuntive), con un netto calo per k maggiori. Si adotta $k = 2$ in entrambe le configurazioni.

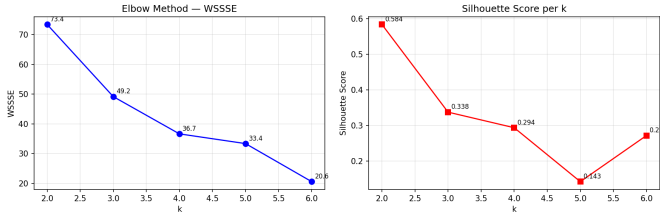


Figura 2. Schema Elbow - Studio base

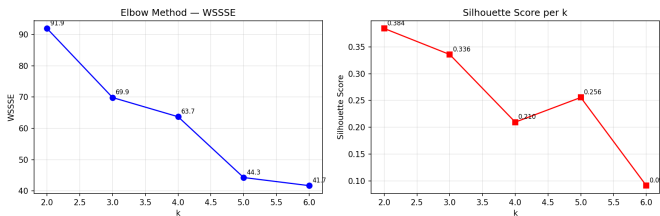


Figura 3. Schema Elbow - Studio esteso

VI. DEPLOYMENT

Il deployment è stato effettuato tramite containerizzazione dei nodi del sistema con **Docker**, simulando in questo modo un cluster di macchine distinte. Ogni nodo dell'architettura corrisponde a un container Docker collegato a una stessa Docker Network. L'orchestrazione dell'intero cluster è gestita tramite **Docker Compose**: un file descrive in modo dichiarativo tutti

i servizi, definendone i parametri di configurazione, tra cui quelli necessari alla corretta comunicazione tra i nodi. La maggior parte dei nodi viene istanziata da immagini pre-costruite reperite dai registry pubblici. Rispetto a un deploy su Cloud con servizi gestiti (come AWS EC2 o AWS EMR), questo approccio manuale offre un livello maggiore di personalizzazione e controllo sui singoli nodi, oltre a un certo valore didattico nello sperimentare la configurazione delle varie tecnologie. Eseguendo tutti i container sulla stessa macchina porta i nodi inevitabilmente a contendersi un insieme ristretto di risorse computazionali, condividendo il medesimo hardware, e ad avere una latenza di comunicazione pari a zero, lontana da un caso reale.

VII. VALUTAZIONE DEI RISULTATI

A. Query

Le performance e le statistiche delle query sono state misurate da prima della lettura del dataset fino alla generazione del risultato. Come già detto in precedenza, il job principale per lo studio dell'esecuzione delle query ci permette di eseguire un numero di run consecutive per ciascuna versione delle tre query. Tramite questo tipo di analisi è possibile notare come Spark utilizzi un'ottimizzazione delle risorse a runtime, abbassando il tempo di processamento di un'esecuzione consecutiva di una stessa query.

1) *Q1*: La Q1 evidenzia che AA presenta un cancellation rate superiore a DL in tutti i mesi, con picco a gennaio per entrambe le compagnie: poiché tale mese non registra il volume di traffico più elevato, l'aumento delle cancellazioni è verosimilmente legato a condizioni operative specifiche. Il minimo si osserva a febbraio, in particolare per DL. Il ritardo medio in partenza è sempre maggiore per AA, con andamenti opposti tra le due compagnie (AA in crescita da gennaio ad aprile, DL in calo). Gli outlier sono nettamente più marcati per AA (ritardi massimi oltre 3000 minuti) rispetto a DL (circa 1100–1200 minuti), mentre i valori minimi negativi confermano la presenza di voli partiti in anticipo.

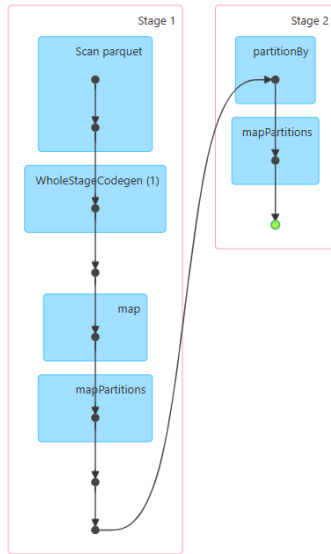


Figura 4. DAG - Query 1 RDD

2) *Q2*: La Q2 mostra che le compagnie con il ritardo medio in arrivo più elevato non coincidono con le più grandi: OH registra il valore massimo (circa 17.16 minuti), seguita da F9 e G4, mentre DL e UA, pur con molti voli, presentano ritardi inferiori. Le componenti dominanti del ritardo sono LA-TE_AIRCRAFT_DELAY e CARRIER_DELAY, riconducibili alla propagazione dei ritardi degli aeromobili precedenti e a fattori operativi interni; il SECURITY_DELAY è trascurabile e il NAS_DELAY contribuisce in misura minore.

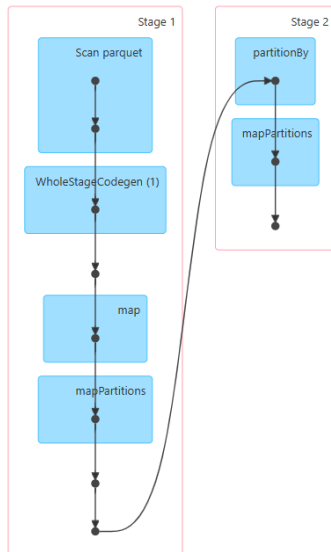


Figura 5. DAG - Query 2 RDD

3) *Q3*: La Q3 analizza la distribuzione del ritardo in partenza per ora del giorno (percentili 25°–90° stimati con t-digest) per AA, DL, UA e WN. Tutte le compagnie mostrano un andamento temporale comune: ritardi minimi al mattino (valori in prevalenza negativi), crescita progressiva e picco serale (circa 18–22), coerente con la propagazione dei ritardi. Per

AA, DL e UA la mediana resta prossima allo zero anche di sera, mentre il 90° percentile sale fino a 60–70 minuti, segnalando un peggioramento concentrato nella coda della distribuzione. WN si distingue per una mediana positiva già dal primo pomeriggio, indice di un ritardo più sistematico. Il range complessivo conferma gli outlier di AA (massimo 3403 minuti, contro 1349 di UA, 1222 di DL e 721 di WN), con minimi negativi per tutte le compagnie.

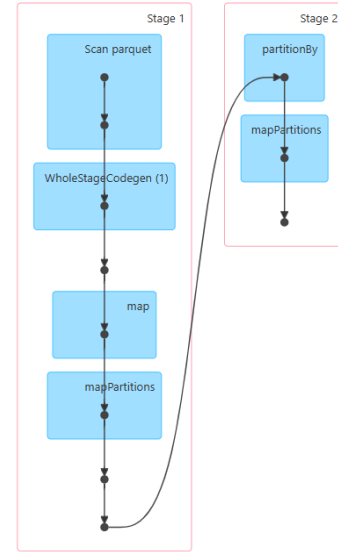


Figura 6. DAG - Query 3 RDD - valido sia per KLL che per t-digest

4) *Analisi dei tempi di esecuzione delle query*: Di seguito sono riportate le tabelle con le statistiche relative ai tempi di esecuzione delle varie versioni di ciascuna query, calcolate su 100 run:

Tabella II
QUERY 1 - TEMPI DI ESECUZIONE CON 1 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.1212	2.0399	0.1758	0.1445	0.0371
sql	0.1230	2.0249	0.1797	0.1479	0.0367
RDD	6.5016	8.0752	6.6282	6.6085	0.0242

Tabella III
QUERY 1 - TEMPI DI ESECUZIONE CON 2 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.1124	3.1776	0.1845	0.1379	0.0942
sql	0.1101	2.9267	0.1865	0.1420	0.0811
RDD	5.4328	7.8719	5.5791	5.5410	0.0612

Tabella IV
QUERY 1 - TEMPI DI ESECUZIONE CON 3 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.1164	3.1691	0.1982	0.1481	0.0957
sql	0.1184	3.2022	0.1976	0.1462	0.0959
RDD	3.9163	7.3384	4.2562	4.2018	0.1340

Tabella V
QUERY 2 - TEMPI DI ESECUZIONE CON 1 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.4502	2.5078	0.5045	0.4695	0.0430
sql	0.4594	2.4635	0.5151	0.4811	0.0408
RDD	10.7304	12.8162	10.8780	10.8466	0.0587

Tabella VI
QUERY 2 - TEMPI DI ESECUZIONE CON 2 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.3195	3.3157	0.4050	0.3495	0.0925
sql	0.3171	3.172	0.4115	0.3683	0.0824
RDD	7.8331	9.9593	8.1484	8.0939	0.0764

Tabella VII
QUERY 2 - TEMPI DI ESECUZIONE CON 3 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.2510	3.4457	0.3428	0.2859	0.1048
sql	0.2480	3.5560	0.3464	0.2876	0.1123
RDD	5.6024	8.6941	5.9228	5.8259	0.1176

Tabella VIII
QUERY 3 - TEMPI DI ESECUZIONE CON 1 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	1.4427	4.3182	1.5766	1.5400	0.0842
sql	1.4752	4.3279	1.6100	1.5619	0.0822
RDD	4.5384	5.5291	4.6541	4.6330	0.0122
KLL	5.5001	7.3334	5.6856	5.6604	0.0327
t-digest	12.5823	14.5635	12.7613	12.7184	0.0515

Tabella IX
QUERY 3 - TEMPI DI ESECUZIONE CON 2 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.9856	4.7753	1.1557	1.0971	0.1474
sql	0.9593	4.6233	1.1485	1.0804	0.1404
RDD	2.8263	5.6113	3.0267	2.9930	0.0743
KLL	3.4318	7.3593	3.6928	3.6560	0.1561
t-digest	7.4905	9.7371	7.6470	7.6155	0.0525

Tabella X
QUERY 3 - TEMPI DI ESECUZIONE CON 3 SPARK WORKER

API	min [s]	max [s]	avg [s]	mediana [s]	varianza [s]
df	0.9469	5.4276	1.1896	1.0960	0.2125
sql	0.9607	5.4056	1.1784	1.0924	0.2053
RDD	2.5713	6.2173	2.7626	2.7101	0.1333
KLL	3.1078	6.4609	3.3031	3.2557	0.1084
t-digest	6.3593	9.9995	6.5991	6.5523	0.1271

Per quanto concerne la Query 3 i tempi superiori (a prescindere dal numero di Spark Worker) di KLL e t-digest rispetto all'implementazione non approssimata in RDD non sono inattesi: trattandosi di algoritmi di approssimazione basati su sketch, il loro vantaggio emerge solo su una mole di dati molto elevata e in sistemi realmente distribuiti, dove il costo del calcolo diventerebbe proibitivo. Anche all'aumentare del numero di Spark Worker, in un contesto locale, con dataset contenuto e risorse limitate, come nel nostro caso, l'overhead

di costruzione, serializzazione e merge delle strutture probabilistiche prevale sul beneficio, rendendoli più lenti rispetto a un'implementazione esatta in RDD.

B. Clustering

1) *Principal Component Analysis*: Poiché lo spazio delle feature è multidimensionale, i cluster sono stati visualizzati proiettando le compagnie aeree sulle prime due componenti principali tramite **Principal Component Analysis (PCA)**, che spiegano il 68.3% della varianza nello studio base e il 60.2% nello studio esteso: la rappresentazione è quindi fedele a circa due terzi dell'informazione. Ogni punto rappresenta una compagnia e il colore rappresenta il cluster a cui quest'ultima è stata assegnata. Questa proiezione evidenzia OH come outlier nettamente isolato (a causa di forti ritardi e presenza massiccia di cancellazioni). Nello studio base $k = 2$ di fatto separa questo singolo outlier dal resto, rendendo questo tipo di studio non propriamente interessante. Discorso diverso per lo studio esteso, dove la suddivisione è più bilanciata e raccoglie al meglio la natura delle compagnie.

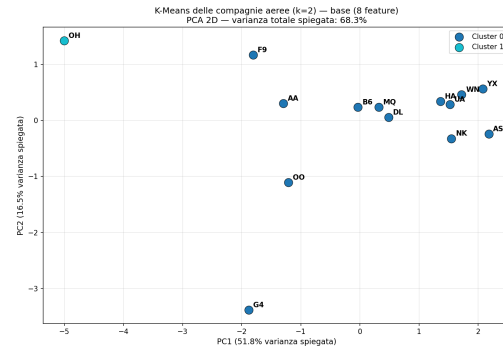


Figura 7. PCA - Studio base

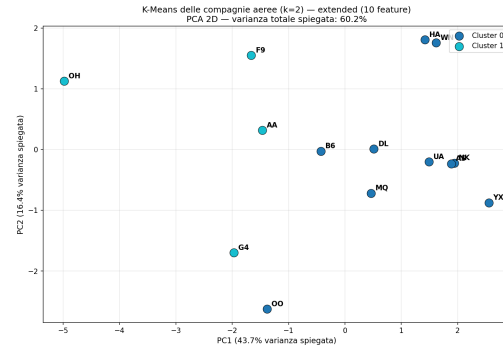


Figura 8. PCA - Studio esteso

2) *Profiling dei cluster*: Per attribuire un significato ai cluster è stata prodotta una **heatmap** dei centroidi espressi in **z-score** per feature (rosso = sopra la media globale, blu = sotto la media globale). Nello studio base il Cluster 0 (13 carrier) presenta valori prossimi alla media, mentre il Cluster 1 (il solo OH) è fortemente positivo su tutte le dimensioni operative. Nello studio esteso i due gruppi assumono una lettura gestionale più chiara: il Cluster 0 (che contiene 10

compagnie) raggruppa le compagnie virtuose (z-score negativi sui ritardi, con on-time rate leggermente positivo), mentre il Cluster 1 (che invece contiene 4 compagnie) raccoglie le compagnie con disservizio operativo diffuso, positive su ritardi, cancellazioni e voli devianti.

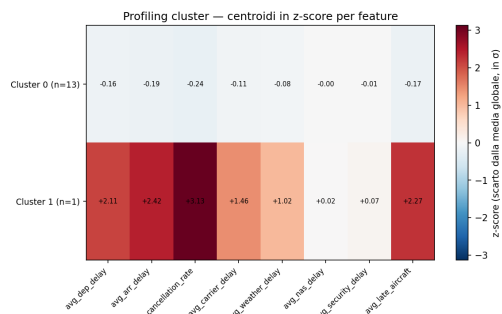


Figura 9. z-score - Studio base

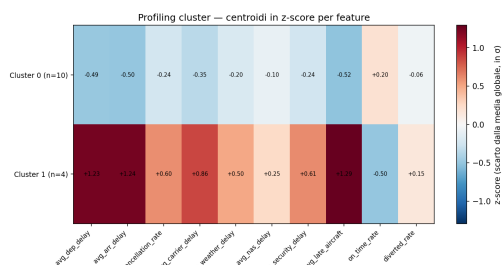


Figura 10. z-score - Studio esteso

3) *Confronto e conclusioni:* Lo studio base ottiene una Silhouette superiore (0.584 contro 0.384), ma tale valore è gonfiato dall'isolamento di un unico outlier e produce una partizione poco informativa (13 compagnie contro 1 sola compagnia). Lo studio esteso, pur ottenendo una Silhouette inferiore, produce invece una segmentazione più equilibrata e interpretabile. L'introduzione di feature aggiuntive consente di descrivere meglio il profilo operativo delle compagnie, includendo non solo ritardi e cancellazioni, ma anche regolarità del servizio e voli dirottati. I cluster ottenuti assumono quindi un significato più chiaro: il Cluster 0 raccoglie compagnie caratterizzate da valori più contenuti di ritardo e cancellazione e da un comportamento complessivamente più regolare, mentre il Cluster 1 raggruppa compagnie con criticità operative più marcate, associate a ritardi medi più elevati, maggiore cancellation_rate e minore on_time_rate. Si conclude quindi che lo studio base fornisce una separazione geometricamente più netta, ma meno informativa dal punto di vista dell'analisi. Lo studio esteso, invece, riduce la separabilità misurata dalla Silhouette, ma migliora la rilevanza operativa dei cluster, che rappresenta l'obiettivo principale dell'analisi.

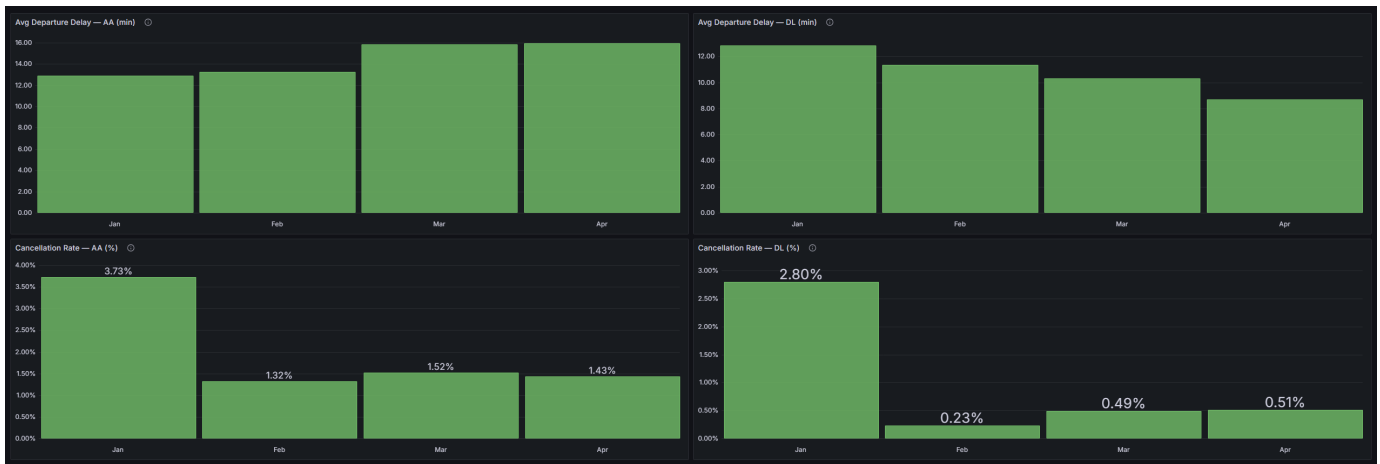


Figura 11. Grafici per il confronto dell'andamento per AA e DL, considerando il valor medio di DEP DELAY e il CANCELLATION RATE aggregati su base mensile

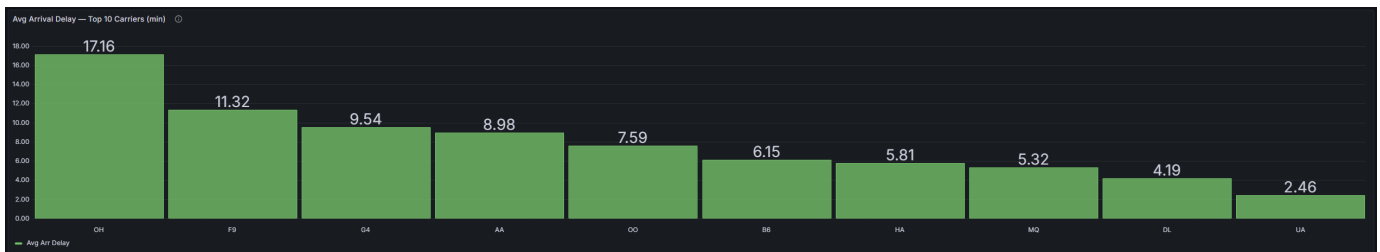


Figura 12. Grafico che mostrano la classifica delle 10 compagnie in base al ritardo medio in arrivo



Figura 13. Grafico che consenta di confrontare, per tali compagnie, l'incidenza media delle diverse cause di ritardo.

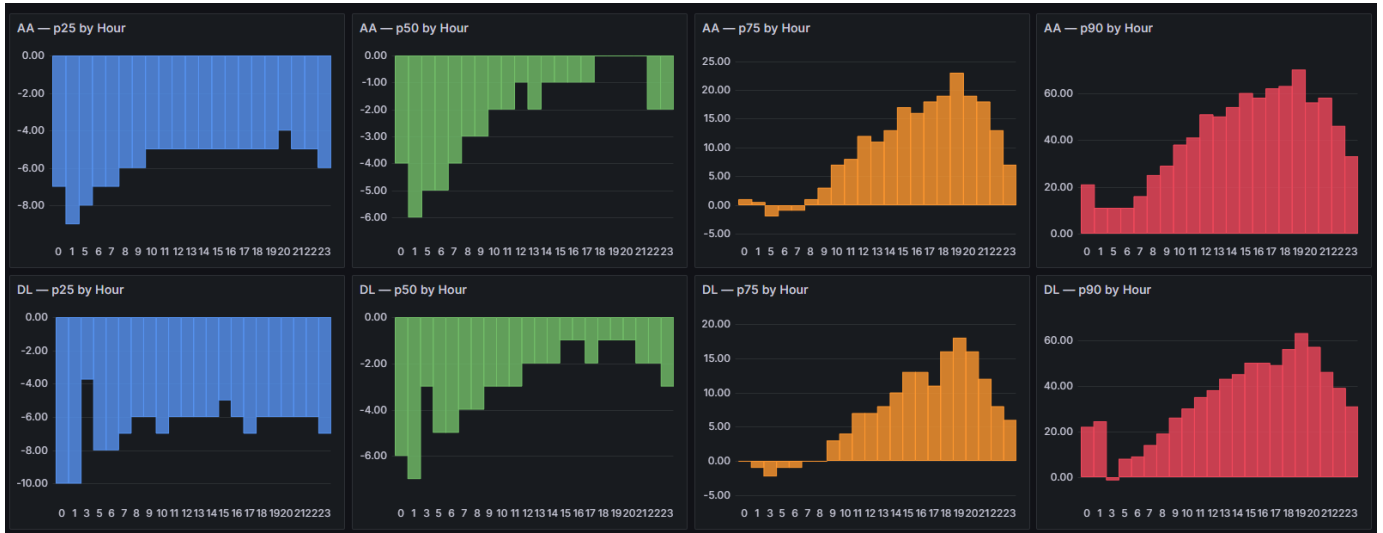


Figura 14. Grafici che consentano di confrontare, per ciascuna compagnia, la distribuzione dei ritardi nelle diverse fasce orarie. (1/2)

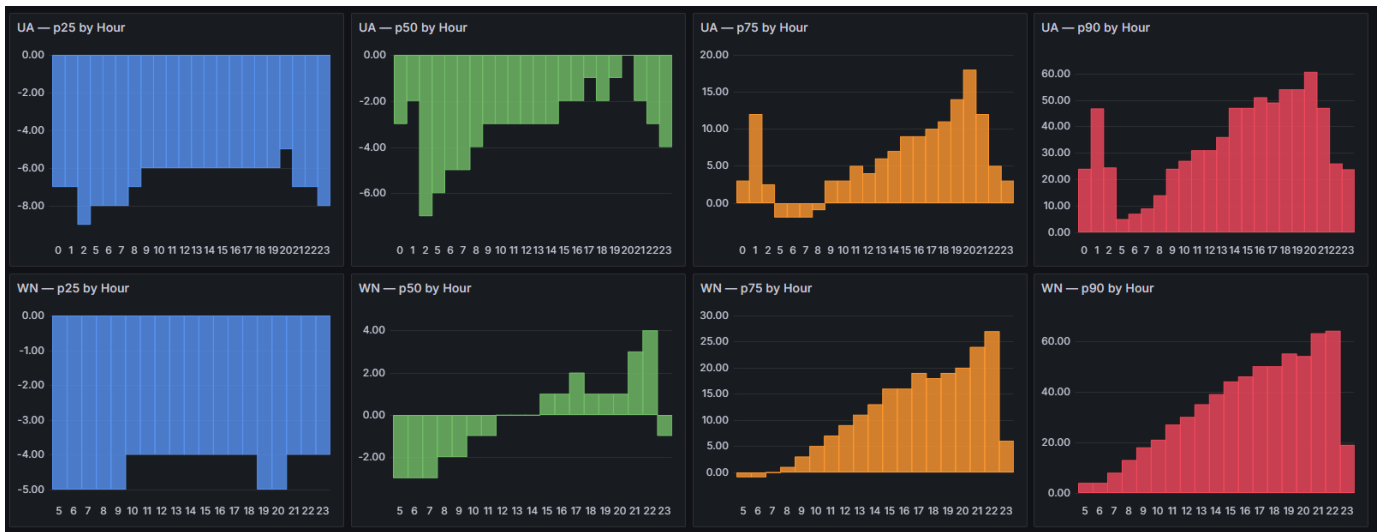


Figura 15. Grafici che consentano di confrontare, per ciascuna compagnia, la distribuzione dei ritardi nelle diverse fasce orarie. (2/2)